IMDb Project Report

Data Mining, Foundations (A.Y. 2024 - 2025)

Francesco Curia, Francesco Secoli

Contents

1 Data understanding and preparation **1**

1.1 Casting and encoding 1

1.2 Feature creation, correlation and elimination 2

1.3 Outlier handling and data transformation 4

1.4 Missing values and `originalTitle` standardization 5

1.5 Final results 7

2 Clustering **8**

2.1 K-means 8

2.2 Bisecting K-means 9

2.3 DBSCAN 11

2.4 Hierarchical clustering 12

3 Classification **13**

3.1 Binary classification 14

3.2 Multiclass classification 16

4 Regression **17**

4.1 Univariate and multiple regression 17

4.2 Multivariate regression 18

5 Pattern mining **19**

5.1 Frequent patterns extraction 19

5.2 Rules extraction and target prediction 20

1 | Data understanding and preparation

The dataset is a subset of the IMDb database, one of the most famous online platforms for movies, TV series, and other similar content. It consists of 23 features, which are summarized in **Table 1.4**[1] and is split into two subsets: the `train_set` (16,431 records) and the `test_set` (5,478 records).

This chapter combines data understanding and preparation due to their iterative nature. The transformations were applied consistently to both the `train_set` and `test_set`.

Feature	Description	Data Type
`awardNEW`	Number of award nominations excluding wins	Ratio
`awardWins`	Number of awards the title won	Ratio
`bestRating`	Best title rating	Categorical
`canHaveEpisodes`	Whether or not the title can have episodes	Binary
`countryOfOrigin`	Country where the title was produced	Categorical
`criticReviewsTotal`	Total number of critic reviews	Ratio
`endYear`	End year of a title	Ordinal
`genres`	Genre(s) of the title (e.g., drama, comedy ...)	Categorical
`isAdult`	Whether or not the title is for adults	Binary
`isRatable`	Whether or not the title can be rated by users	Binary
`numRegions`	Number of regions in which the title is available	Ratio
`numVotes`	Number of ratings the title has received	Ratio
`originalTitle`	Title of the product	Categorical
`rating`	IMDb title rating class	Ordinal
`ratingCount`	Number of user ratings submitted for the title	Ratio
`runtimeMinutes`	Runtime of the title in minutes	Ratio
`startYear`	Release year of a title	Ordinal
`titleType`	Type of the title (e.g., movie, short, tvseries...)	Categorical
`totalCredits`	Total number of credits for the title	Ratio
`totalImages`	Total number of images for the title	Ratio
`totalVideos`	Total number of videos for the title	Ratio
`userReviewsTotal`	Total number of user reviews	Ratio
`worstRating`	Worst title rating	Categorical

Table 1.1: description and data types of the dataset's features, listed in alphabetical order. The column `awardNominationsExcludeWins` has been renamed to `awardNEW` in the table to optimize space.

1.1 | Casting and encoding

1.1.1 | Casting (`rating`)

A few casting operations were performed in Python to ensure dataset uniformity. Variables were converted from `float64` to `int64`, and `boolean` values were transformed into `0/1`. The most significant change was to the `rating` column, which was originally represented as intervals (e.g., `(7, 8]`). The upper bound of each interval (`x+1`) was used as the representative value, transforming the column into a categorical variable with integer values from 1 to 10.

[1] The data types listed in the **Data type** column do not directly correspond to Python's `pandas` data types. There is no one-to-one mapping between Python's type system and the dataset's data types. For instance, both `startYear` and `runtimeMinutes` are stored as `int`, but they are treated as categorical and ratio types, respectively.

1.1.2 | Encoding (`titleType`, `countryOfOrigin`, `genres`)

To handle categorical variables, encoding techniques were applied to those columns without an intrinsic numerical representation. The encoding strategy depended on the characteristics of each feature.

- The `titleType` column was encoded using **label encoding**, mapping each unique value to an integer (0–8) corresponding to labels like `tvSeries`, `movie`, and others[2].
- For `countryOfOrigin` and `genres`, **one-hot encoding** was applied, and two new features were added to capture the count of elements per entry: `NumberOfCountryOfOrigin` and `NumberOfGenres`.

A challenge was encountered with the `countryOfOrigin` column, as the categorical values differed between the training and test sets (see African countries **Figure 1.1**). To address this issue additional filler columns were introduced with the appropriate names, aligning the features in the `train_set` and `test_set`.

Figure 1.1: maps illustrating the number of titles in the `train_set` (left) and `test_set` (right) dataset for each country of origin. Note that some countries included in the dataset are not represented on the maps due to their historical status, as they no longer exist (e.g., USSR, East Germany, West Germany).

1.2 | Feature creation, correlation and elimination

1.2.1 | Feature creation (`numberOfMedia`, `startYearDecade`, `totalRecognitions`)

`numberOfMedia` was introduced to measure the multimedia content of a record, defined as the sum of `totalImages` and `totalVideos`. Exploratory analysis showed significant asymmetry: 20% of records had no images, and 90% had no videos. `numberOfMedia` is highly correlated with `totalImages` and weakly with `totalVideos` (1 and 0.17, see **Figure 1.4**). Due to the sparsity of `totalVideos` and its high correlation with `totalImages`, both columns were removed.

`totalRecognitions` was introduced to better capture a film's achievements, defined as the sum of `awardNominationsExcludeWins` and `awardWins`. This accounts for cases where a film wins awards but has no recorded nominations, ensuring more accurate categorization. To avoid redundancy, `awardWins` and `awardNominationsExcludeWins` were removed (see **Figure 1.4**).

[2]Note the absence of the `videogame` category, as it has been removed (**Section 1.4.3**).

startYearDecade groups films by their decade of release: this reduced the granularity of **startYear**, as shown by comparing the plots in **Figure 1.3**, and allowed for more immediate comparisons between the **train_set** and **test_set**.

Decadal aggregation made it easier to identify trends, as illustrated in **Figure 1.2**, which shows an increasing average rating for **movie**, **video**, and **short** over the decades, while other are less stable.

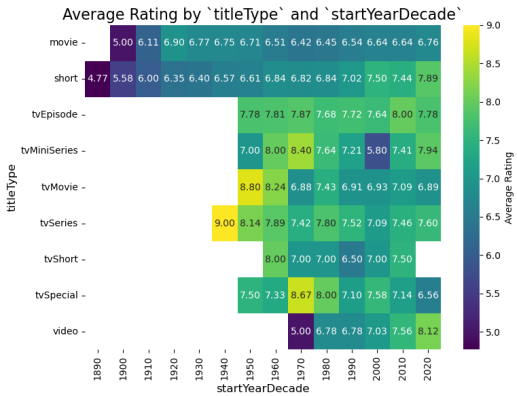


Figure 1.2: mean **rating** by **startYearDecade** and **titleType** of records

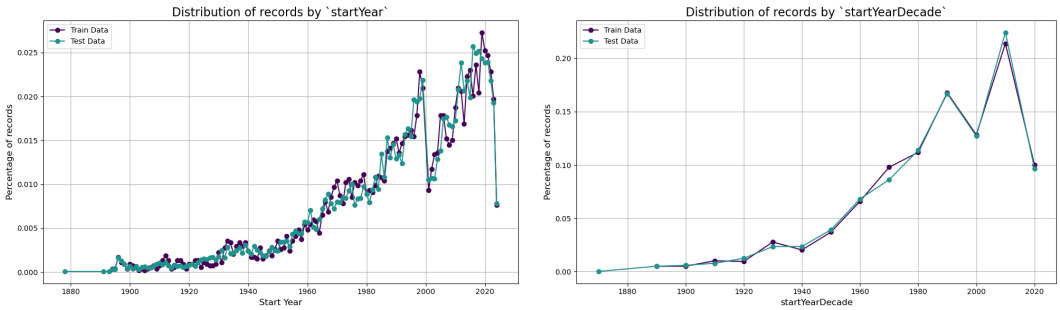


Figure 1.3: percentage of records by **startYear** (left) and by **startYearDecade** (right)

1.2.2 | Feature correlation and elimination

In addition to the features removed due to their high correlation with the newly introduced features, a Pearson correlation analysis (**Figure 1.4**) revealed a strong correlation between **ratingCount** and **numVotes**. As a result, **ratingCount** was removed, as it offered less semantic relevance to the dataset and was deemed less valuable for future analyses.

Further analysis identified three additional columns, **worstRating**, **bestRating**, and **isRatable**, that contained the same values across all entries in training and test sets (1, 0, and **true**, respectively). Since these columns did not provide meaningful information, they were removed from the dataset.

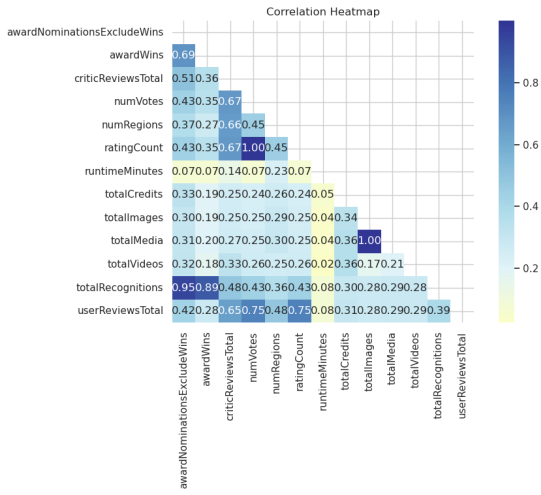


Figure 1.4: Pearson's coefficient of all the ratio features of the dataset

An inconsistency was found between the `isAdult` and `Adult`[3] columns, with a few dozen cases where `isAdult` was `1` and `Adult` was `0`. To resolve this, a "logical OR" operation updated the `Adult` column, and `isAdult` was dropped.

This adjustment affected few entries, but a conservative approach was used to prioritize avoiding false negatives, even at the cost of occasional false positives.

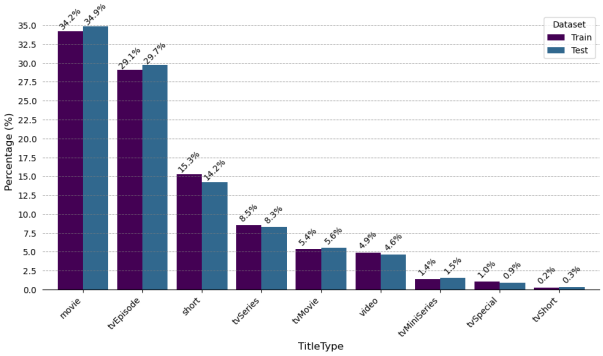


Figure 1.5: percentage of records by `titleType` (train vs test)

Moving from the features to the records of the two datasets, and analyzing the variable `TitleType` (**Figure 1.5**), it is evident that `tvMiniSeries`, `tvSpecial`, and `tvShort` are just residual categories, all below 2%. These categories were considered superfluous and removed from the datasets, as they were deemed noise that could introduce unnecessary complexity.

1.3 | Outlier handling and data transformation

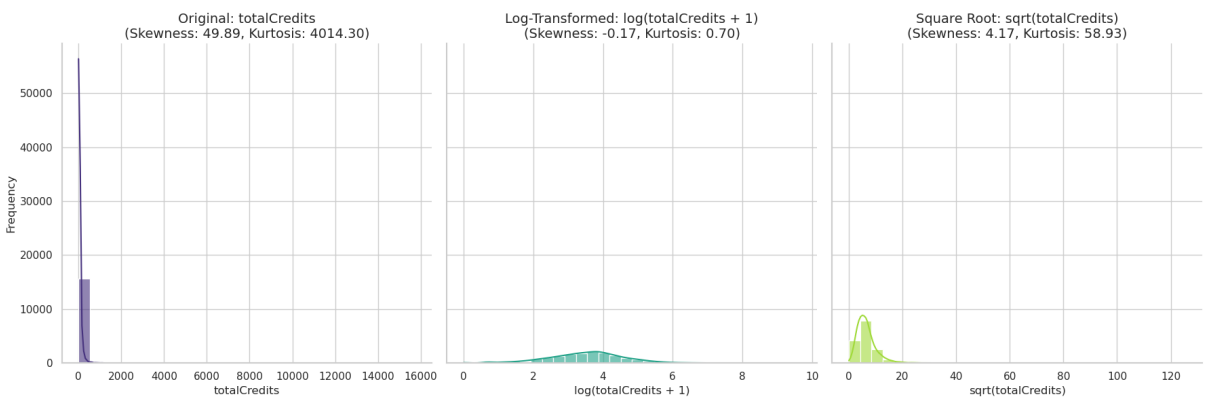


Figure 1.6: visual representation of the original (left), log-transformed (center), and sqrt-transformed (right) distributions of `totalCredits`. Each plot shows both the line plot of the distribution and the histogram bins calculated using Sturges' rule.

The numerical attributes (`criticReviewsTotal`, `numRegions`, `numVotes`, `totalCredits`, `totalMedia`, `totalRecognitions`, `userReviewsTotal`) have highly skewed distributions, with peaks at 0 or 1: only a few titles have many awards, nominations, photos, or reviews, while most have minimal values.

To address skewness and variance, log and sqrt transformations were applied and compared using skewness and kurtosis as evaluation metrics (see **Figure 1.6**). The transformation that achieved the most balanced distribution was then applied to the dataset, as summarized in **Table 1.2**.

[3] Created via one-hot encoding of the `genres` feature.

Feature	original		log-transformed		square root	
	Skewness	Kurtosis	Skewness	Kurtosis	Skewness	Kurtosis
`criticReviewsTotal`	14.68	304.28	**2.44**	**6.55**	4.60	32.42
`numRegions`	4.15	22.19	**1.56**	**1.76**	2.33	6.19
`numVotes`	28.30	980.46	**1.31**	**2.13**	12.49	216.36
`runtimeMinutes`	17.67	904.78	**-0.99**	**0.85**	0.55	7.76
`totalCredits`	50.21	4084.68	**-0.17**	**0.70**	4.13	58.22
`totalMedia`	28.29	1043.79	**1.26**	**1.73**	6.09	74.18
`totalRecognitions`	20.31	626.46	**3.48**	**12.31**	5.05	41.29
`userReviewsTotal`	45.96	3411.08	**2.05**	**4.93**	7.42	105.90

Table 1.2: comparative table of skewness and kurtosis for unnormalized data (original), logarithmic transformation, and square root transformation. Attributes corresponding to the chosen normalization are highlighted in bold blue for clarity.

The `runtimeMinutes` feature shows a slightly different pattern. Its distribution is right-skewed but with a bimodal-like shape at lower values. This distribution likely reflects differences across `titleType` categories, with distinct peaks arising from variations in runtimes. The feature was converted from minutes to hours by dividing it by 60[4], as this provides a more interpretable scale for analysis and mantaining the distribution.

Outliers were detected using the Interquartile Range (IQR=1.5) method, grouped by `titleType` to account for distinct distributions and uniform behavior. For strictly positive features, zeros were considered default and excluded from the IQR calculation. Outliers were identified only for upper values, except for `runtimeMinutes`, where no such behavior was identified[5].

The process was applied uniformly across all features in a single pass to avoid bias. Outliers were identified in the *normalized* dataset, with corresponding rows removed from both the normalized and original datasets to ensure consistency. As a result, 1799 records (11.43%) were eliminated from the train set and 597 records (11.40%) from the test set.

1.4 | Missing values and `originalTitle` standardization

The dataset contains missing values in the columns `endYear`, `genres`, and `runtimeMinutes` (**Figure 1.7**).

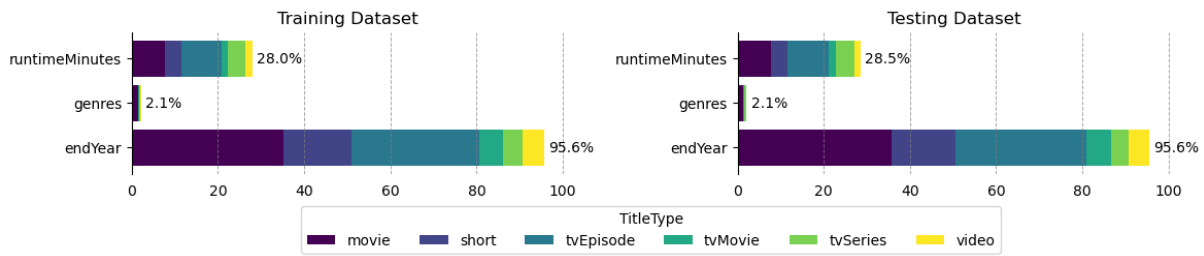


Figure 1.7: global missing values percentage in train set and test set and internally grouped by `titleType`

[4]The name of the column doesn't change
[5]Null values were also removed from this column, as they are handled later in **Section 1.4.3**.

1.4.1 | Missing `endYear`

Analysis of missing values in the `endYear` column (**Figure 1.8**) showed that only `tvSeries` and `tvMiniSeries` had *non-null* entries. Missing values likely stem from either lack of data or series still in production, as suggested by the `startYear` distribution (**Figure 1.9**). Due to the high number of missing values and the lack of reliable imputation methods, the `endYear` column was dropped.

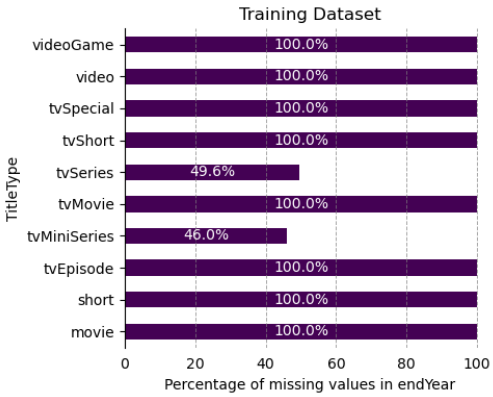


Figure 1.8: missing values in `endYear`, grouped by `titleType`. The chart shows only the `train_set`, but the results are essentially identical for the `test_set`.

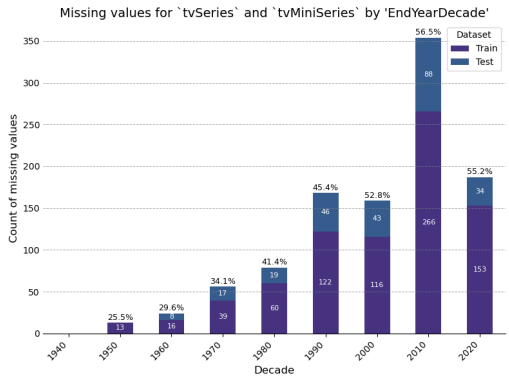


Figure 1.9: missing values in `endYear` for `tvSeries` and `tvMiniSeries` `titleType` grouped by `startYearDecade`. The percentage are on the junction of `test_set` and `train_set`

1.4.2 | Missing `genres`

A minimal number of entries remained unclassified, so a new `unknown` genre category was introduced during data preparation. This addition did not unbalance the dataset, as its percentage was comparable to smaller genres like `Sci-Fi` and `Adult`, and far lower than major genres like `Drama` (36%) or `Crime` (10%).

1.4.3 | Missing `runtimeMinutes`

To address missing values in the `runtimeMinutes` column, entries from the `Videogame` genre, where over 99% of values were missing, were dropped. The presence of runtime values in this genre appeared erroneous, and `Videogame`, being fundamentally different from other `titleTypes`, has an undefined or irrelevant runtime. Since it represented a small fraction of the dataset, its exclusion had minimal impact. For remaining entries, missing values were imputed using the *median* of each `titleType` group. The median was chosen for its robustness to outliers, ensuring the imputation was not affected by extreme values.

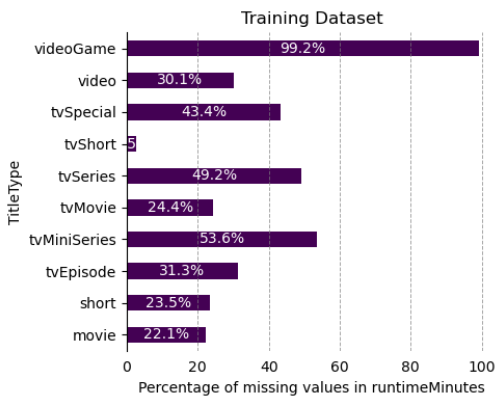


Figure 1.10: percentage of missing values in `runtimeMinutes` by `titleType`.

1.4.4 | originalTitle standardization

The originalTitle column was normalized by removing special characters and converting all text to lowercase. Manual inspection revealed that many titles included the word "episode" or similar. An analysis of the most frequent k-grams (with $k = 4, 5, 6$) identified the 5-gram "pisod" as the second most frequent. A correlation analysis between rows containing the 5-gram "pisod" and the titleType column confirmed that the corresponding categories were plausible. Therefore, no modifications were made to these entries.

Table 1.3: number of 5-gram "pisod" in originalTitle, grouped by titleType.

titleType	train	test
tvEpisode	516	169
video	1	0
short	0	1

1.5 | Final results

As shown in **Figure 1.11**, Pearson's correlation reveals the strongest relationships among features related to evaluations, such as criticReviewsTotal, numVotes, and totalMedia, highlighting shared dynamics in audience engagement and critical reception.

In contrast, runtimeMinutes shows weak correlations with other variables, suggesting it is largely independent of reviews or awards. Moderate correlations between totalCredits and numVotes suggest that larger productions or those with more promotional efforts tend to attract higher audience engagement.

For the analysis, four datasets were prepared: train_set and test_set, which include all transformations except for those described in **Section 1.3**, and transformed_train_set and transformed_test_set, where all transformations were applied.

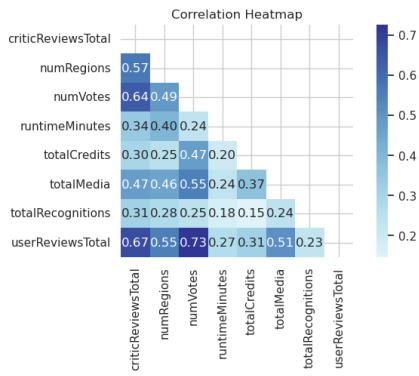


Figure 1.11: Pearson's correlation heatmap for final train_set normalized after data understanding and preparation.

Table 1.4: Statistical summary of transformed_train_set and transformed_test_set (values rounded to two decimal places): countryOfOrigin and genres represent one-hot encoded features. The total column count includes both the one-hot encoded columns and an additional column indicating the original cardinality before encoding.

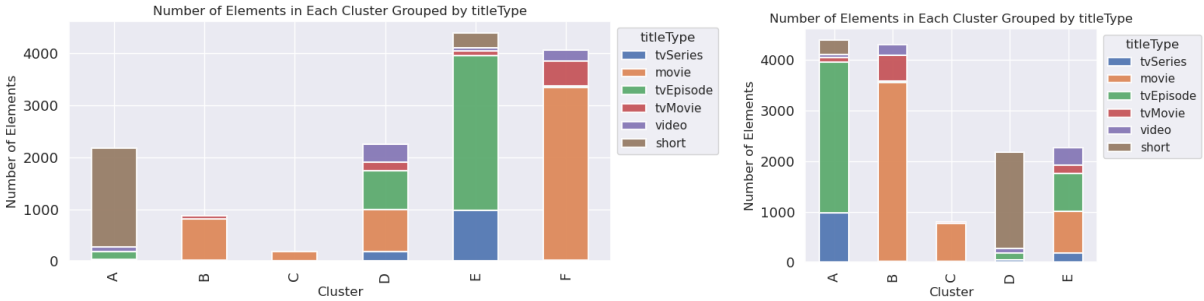
Feature	Data Type	New	Columns	Train (13.943 records)					Test (4.641 records)				
				Min	Max	Mean	Std	Median	Min	Max	Mean	Std	Median
countryOfOrigin	Categorical	✓	162	-	-	-	-	-	-	-	-	-	-
criticReviewsTotal	Ratio		1	0.00	4.81	0.40	0.78	0.00	0.00	5.15	0.42	0.80	0.00
genres	Categorical	✓	30	-	-	-	-	-	-	-	-	-	-
numRegions	Ratio		1	0.69	3.83	1.08	0.63	0.69	0.69	3.58	1.10	0.64	0.69
numVotes	Ratio		1	1.79	9.61	3.93	1.52	3.56	1.79	9.61	3.95	1.54	3.61
originalTitle	Categorical		1	-	-	-	-	-	-	-	-	-	-
rating	Ordinal		1	1	10	7.12	1.40	7	1	10	7.12	1.39	7
runtimeMinutes	Ratio		1	0.00	2.93	0.92	0.57	0.83	0.02	2.87	0.94	0.57	0.90
startYear	Ordinal		1	1900	2025	1995	15.56	2000	1900	2025	1995	15.56	2000
startYearDecade	Categorical	✓	1	-	-	-	-	-	-	-	-	-	-
titleType	Categorical		1	-	-	-	-	-	-	-	-	-	-
totalCredits	Ratio		1	0.00	7.54	3.45	1.05	3.56	0.00	7.82	3.48	1.03	3.56
totalMedia	Ratio	✓	1	0.00	6.03	1.22	1.07	0.69	0.00	5.96	1.25	1.08	0.69
totalRecognitions	Ratio	✓	1	0.00	3.91	0.20	0.54	0.00	0.00	4.01	0.20	0.54	0.00
userReviewsTotal	Ratio		1	0.00	5.69	0.62	0.93	0.00	0.00	5.27	0.63	0.93	0.00
Total	**Dataset**	-	**205**	-	-	-	-	-	-	-	-	-	-

2 | Clustering

The original dataset was preferred for clustering over the log-transformed one, as the latter compressed scales and reduced interpretability, leading to poorer results. Despite this, the original dataset's skewed distributions impacted density-based algorithms, often resulting in one dominant cluster and several small or singleton clusters.

All ratio features were used (see **Table 1.4**), followed by a subset to test if fewer variables could achieve similar results. Data were normalized with Min-MaxScaler for more meaningful outcomes.

2.1 | K-means



(a) 6-means, all ratio features.

(b) 5-means, selected features.

Figure 2.1: clustering results grouped by `titleType`: (a) all ratio features, and (b) selected features.

The k-means analysis evaluated the silhouette score and SSE for k values ranging from 2 to 29, aiming to identify an "elbow" in the SSE curve that aligned with stable silhouette scores. The upper limit of 29 was chosen based on the number of `genres` categories.

As shown in **Figure 2.2**, an elbow is observed at $k = 6$, where the silhouette score stabilizes and gradually increases, reaching a local maximum at $k = 11$. In $k = [6; 11]$, the most meaningful clustering results were obtained at $k = 6$, where k-means effectively categorized the dataset by `titleType` with minimal outliers. At higher values of k, larger clusters such as *movie* and *series* were split, while other categories remained well-separated.

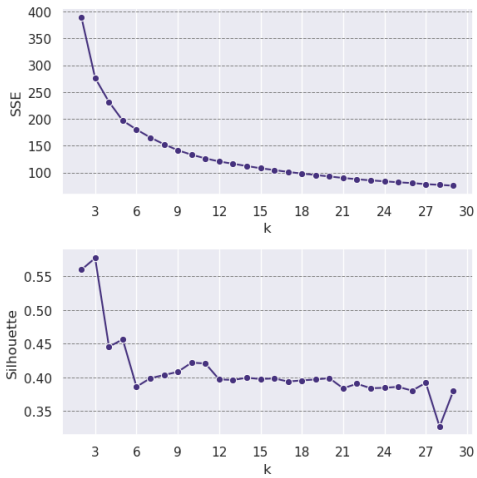


Figure 2.2: SSE (up) and silhouette (down) plots for k-means, with $k = [2; 29]$ for all ratio features in `train_set`

The clustering results, shown in **Figure 2.1a**, highlight semantic relationships between certain categories. For example, `movie` and `tvMovie`, as well as `tvEpisodes` and `tvSeries`, naturally group together, reflecting connections in their data distributions. Conversely, `short` emerges as a distinct and well-isolated category.

Meanwhile, `video` is dispersed across multiple clusters, likely due to its low frequency in the dataset and its heterogeneous characteristics, which align it with various groups based on specific record attributes. An analysis of the centroids, shown in **Figure 2.3**, was conducted to identify a subset of features that could better unify clusters containing similar categories. The goal was to explore whether reducing the dimensionality of the data could enhance clustering results and better align with the analysis objectives.

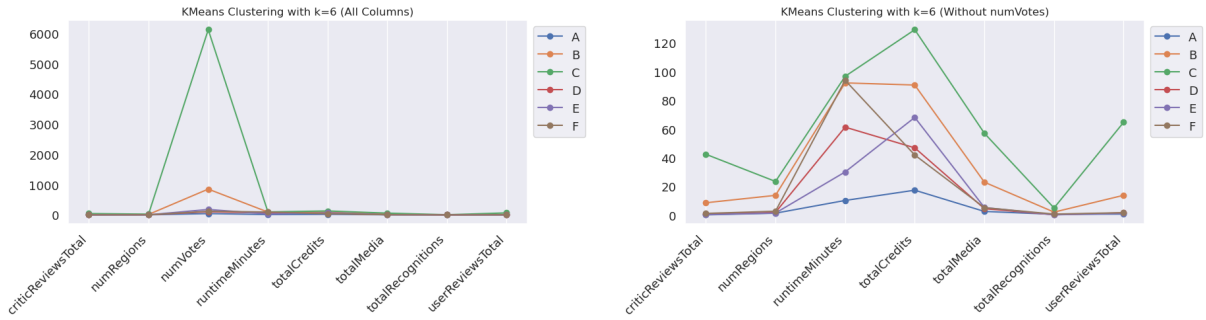


Figure 2.3: parallel line chart with all ratio features for $k = 6$. Centroids are shown for all features (left), and for all but `numVotes` (right) to improve interpretability due to its out-of-scale value.

The primary goal was to identify features that effectively distinguished Cluster A from the others while maintaining the separation of the `short` category. Additionally, the objective was to unify Clusters D and E to group `tvSeries` and `tvEpisodes`, and to consolidate `movie` and `tvMovie` into a single group (Clusters B, C, F). This restructuring sought to create a more coherent and semantically meaningful classification, refining the initial unsupervised clustering results. Through exploratory testing, `numRegions`, `runtimeMinutes`, `totalCredits`, and `totalRecognitions` emerged as the most effective. Their incorporation not only improved clustering performance but also resulted in better separation and alignment, as shown in **Figure 2.4**.

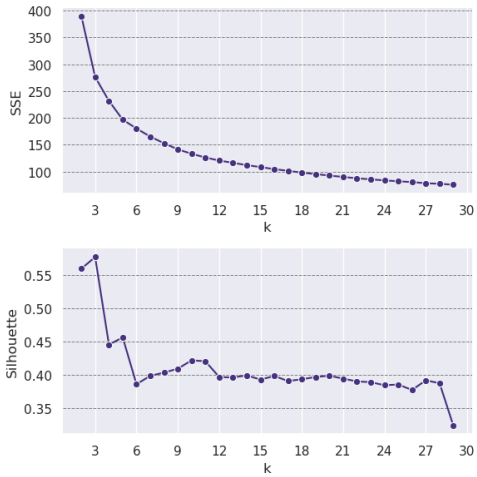
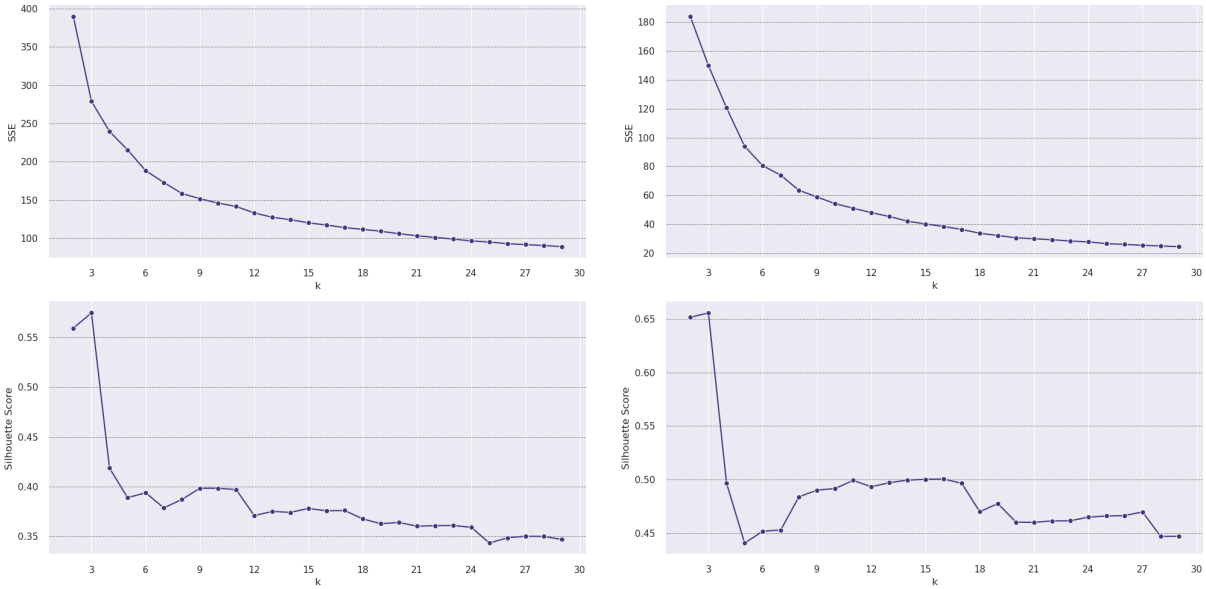


Figure 2.4: SSE (up) and silhouette (down) plots for k-means, with $k = [2; 29]$ for selected features in `train_set`

This refinement reduced the number of clusters to $k = 5$ while preserving nearly the same categorization. The only significant change was observed in the `movie` category, which is now distributed across fewer clusters, as illustrated in **Figure 2.1b**.

2.2 | Bisecting K-means

Bisecting k-means was applied using both all available features and the subset identified in k-means to compare results. While the number of significant clusters in terms of SSE and silhouette is comparable (compare **Figure 2.5a**/**2.2**), bisecting k-means proved less effective in separating the `titleType` categories.



(a) all ratio features. **(b)** selected ratio features.

Figure 2.5: SSE (top) and silhouette (down) plots for bisecting k-means clustering with $k = [2; 29]$.

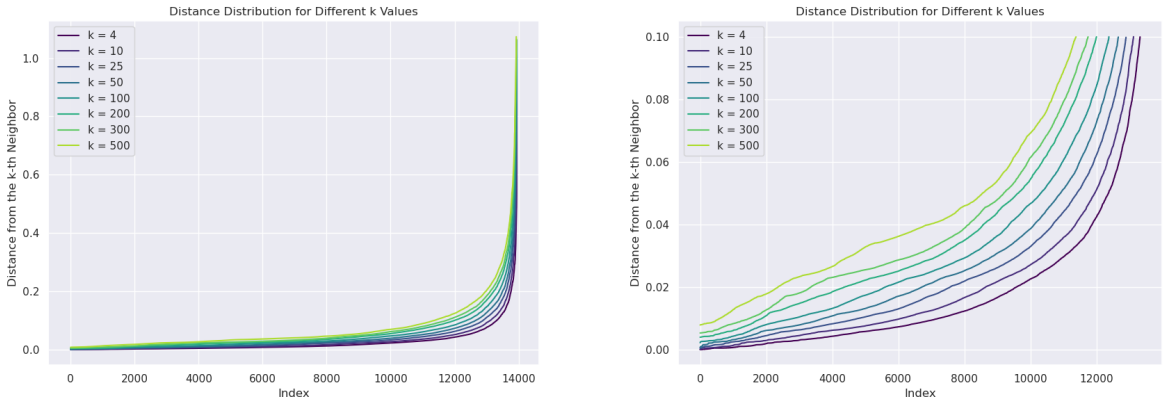
Specifically, for $k = 6$ (**Figure 2.6a**), it failed to distinguish `short` from `tvSeries/tvEpisode`, likely due to the weighting of `runtimeMinutes` over other features. However, for $k = 7$ (**Figure 2.6b**), it managed to achieve better separation. Furthermore shows an excessive fragmentation of the `movie` category into smaller clusters, which, although relevant for clustering, was not aligned with the original analytical goals. A possible explanation for this difference lies in the hierarchical nature of bisecting k-means, which iteratively splits data into subgroups starting from a main cluster. While this approach works well for identifying clusters with an inherent hierarchical structure, it may be less suited for data that are better separated into distinct groups from the outset, as in the case of `titleType`. Additionally, the splitting method used in each iteration optimizes the clustering criterion locally without ensuring optimal global separation between main categories.

The behavior observed in bisecting k-means with the selected features, similar to that of k-means with $k = 5$, suggests that choosing a targeted subset of features led to better separation of categories. By using $k = 5$ (**Figure 2.6c**), a clearer and more consistent separation between clusters was achieved.

(a) 6-means, all features **(b)** 7-means, all features **(c)** 5-means, selected features.

Figure 2.6: comparison of bisecting k-means results with different feature sets.

2.3 | DBSCAN



(a) full visualization without threshold **(b)** visualization with a threshold of 0.1

Figure 2.7: K-th nearest neighbor analysis for $k = [4, 10, 25, 100, 200, 300, 500]$.

Figure 2.7a highlights the difficulty in identifying a clear elbow to determine optimal DBSCAN parameters, ε and minPts. Based on **Figure 2.7b**, a promising range for ε was between 0.02 and 0.06.

Since suitable parameters for satisfactory results could not be identified, an exhaustive search was conducted, varying k from 4 to 200 (in steps of 2) and ε from 0.02 to 0.06 (in steps of 0.001). For each combination the number of noise points, the silhouette score, and the number of identified clusters were recorded.

The results often showed negative silhouette scores, while positive values peaked at 0.4, corresponding only to configurations with a single cluster. As expected, the silhouette score proved unsuitable for density-based methods like DBSCAN, being biased toward globular clusters. Despite extensive parameter tuning, no significant results emerged. Low ε and minPts values consistently produced a single dense cluster, with most points classified as noise and a few isolated clusters. Increasing minPts removed smaller clusters without improving quality, while higher ε values caused DBSCAN to merge nearly all points into one large cluster.

The same approach was repeated using features selected in **Section 2.1** , but results did not improve. The high sensitivity of DBSCAN to parameter selection, combined with the dataset's dense structure, made it difficult to extract meaningful information. For this reason, no specific DBSCAN results are presented. Instead, the analysis focuses on the overall behavior of the clustering method, avoiding the inclusion of insignificant results simply for the sake of showing them.

OPTICS, designed to handle varying-density clusters, was also tested during the dataset exploration phase but faced similar issues due to the dataset's compact structure and small distances between points. It either focused solely on the dense region or progressively merged the entire dataset as parameters were adjusted. Rescaling the data using a MinMaxScaler (e.g., from [0, 1] to [0, 10]) did not improve results. For this reason, OPTICS results are omitted for simplicity.

2.4 | Hierarchical clustering

In this final section on clustering, *hierarchical clustering* is employed to validate the observations and results obtained from *K-means/Bisecting K-means* and *DBSCAN*, such as the compactness and the close proximity of data points.

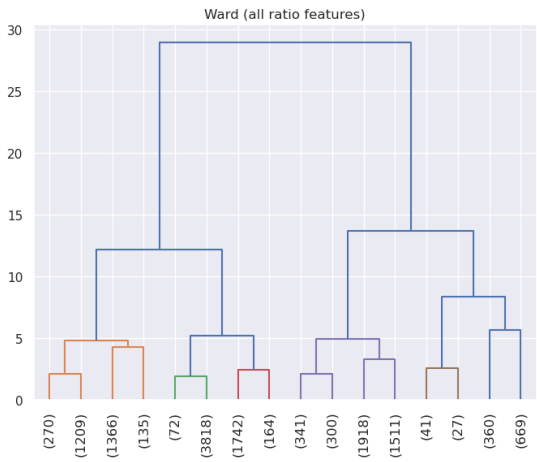
The analysis examines two hierarchical clustering methods, each selected to complement and validate previous clustering results. Ward linkage minimizes intra-cluster variance, creating compact, globular clusters that closely align with the behavior of K-means and Bisecting K-means. This makes it particularly useful for confirming whether the compact clusters identified by these methods reflect the dataset's intrinsic structure, even when clusters vary in size or density.

On the other hand, single linkage prioritizes the proximity of points, merging clusters based on the shortest distance. This method highlights the compactness and closeness of points in dense regions, while also indirectly confirming the limitations of DBSCAN, which struggles with uniformly dense datasets due to its sensitivity to density thresholds.

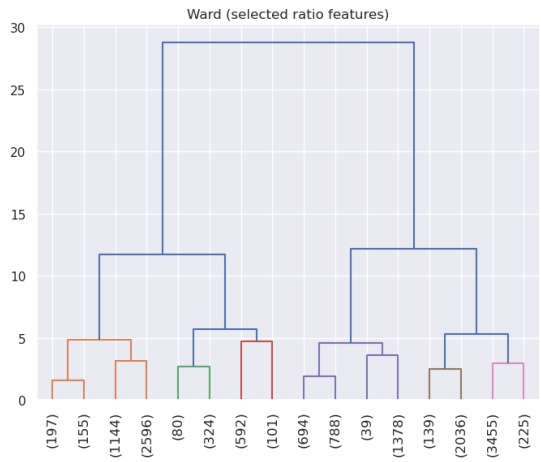
2.4.1 | Validation of K-means through Ward's dendrogram

As shown in **Figure 2.8**, Ward's method produced a clear and well-separated dendrogram for a small number of clusters. This confirms the validity of the K-means clustering approach.

In both dendrograms (all features (2.8a) and selected features (2.8b)) the significant distances between the initial branches leading to four main clusters in each case indicate clear separation and distinctness. These results reflect the outcomes of K-means, which was able to provide meaningful clusters even with a small number of k. Ward's method successfully captures the underlying structure of the data, even with the high-density of this dataset.



(a) clustering results on the full feature set.



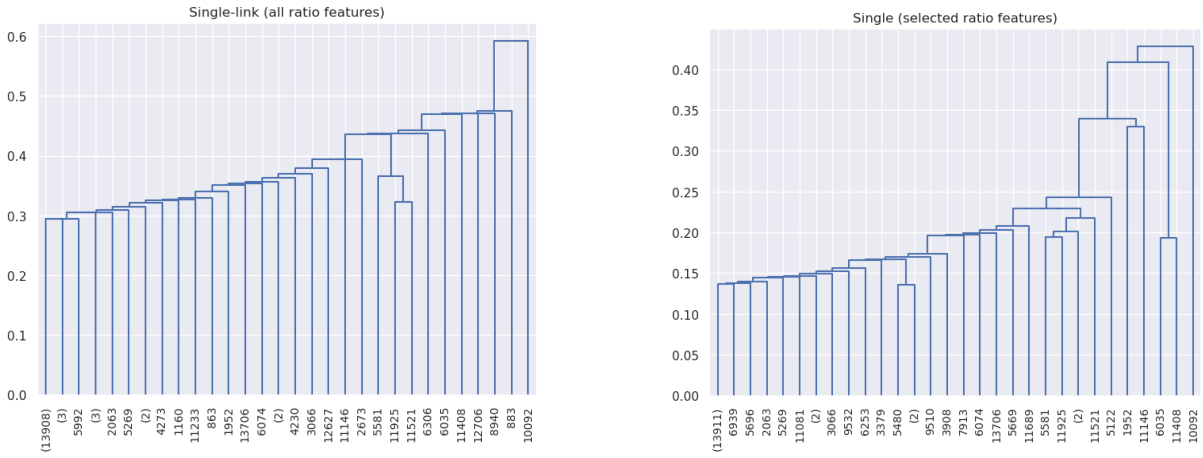
(b) clustering results on the selected features.

Figure 2.8: Ward clustering dendogram.

2.4.2 | Analysis of DBSCAN results and single linkage dendrogram

As previously mentioned, DBSCAN did not yield meaningful clusters. This behavior can be better understood by analyzing the single linkage dendrogram (**Figure 2.9a**), which emphasizes the closest distances between points. The dendrogram shows a tendency to produce many singleton clusters or very small groups[6]. This pattern is consistent across both configurations, with nearly identical dendrograms for all features and selected features.

The single linkage dendrogram highlights the dominance of a single, extremely dense region of points surrounded by sparse areas: when DBSCAN's parameters (ε and minPts) are set to detect this dense region, it identifies it as a single large cluster while classifying remaining points as noise. Increasing ε to include points outside this region merges the entire dataset into one cluster, demonstrating DBSCAN's difficulty in balancing clustering under these conditions.



(a) full feature set

(b) selected features

Figure 2.9: single-linkage clustering. Almost all leaves, except for the leftmost one, represent single points. The leftmost leaf, instead, groups nearly all nodes, highlighting the presence of a single, extremely dense region in the dataset.

3 | Classification

Three classification algorithms (*K-Nearest Neighbors (K-NN)*, *Naive Bayes*, and *Decision Trees*) were evaluated. *K-NN* and *Naive Bayes* were applied to the normalized dataset due to their sensitivity to feature scaling, whereas *Decision Trees*, being scale-invariant, were trained on the unnormalized dataset. In all cases, `train_set` was used for training the models and `test_set` to confirm the results. **Table 3.1** presents the final outcomes for each target variable under the most effective classification strategies, further examined in the subsequent sections.

[6] Additional dendrograms with more branches were generated, but the behavior was consistent with what is shown in the figure and are omitted for simplicity.

Target	Best Model	Accuracy	Precision	Recall	F1 Score
`isMovie`	Decision Tree	0.92	0.90	0.91	0.92
`isHighRated`	Naive Bayes	0.64	0.67	0.66	0.64
`titleType`	Decision Tree	0.86	0.84	0.86	0.84

Table 3.1: Decision Trees outperformed other models for both the binary classification task `isMovie` and the multiclass classification task `titleType`, yielding the best results overall. In contrast, `isHighRated` saw weaker performance across all models, with K-NN and Naive Bayes producing nearly identical outcomes and Decision Trees approaching random selection.

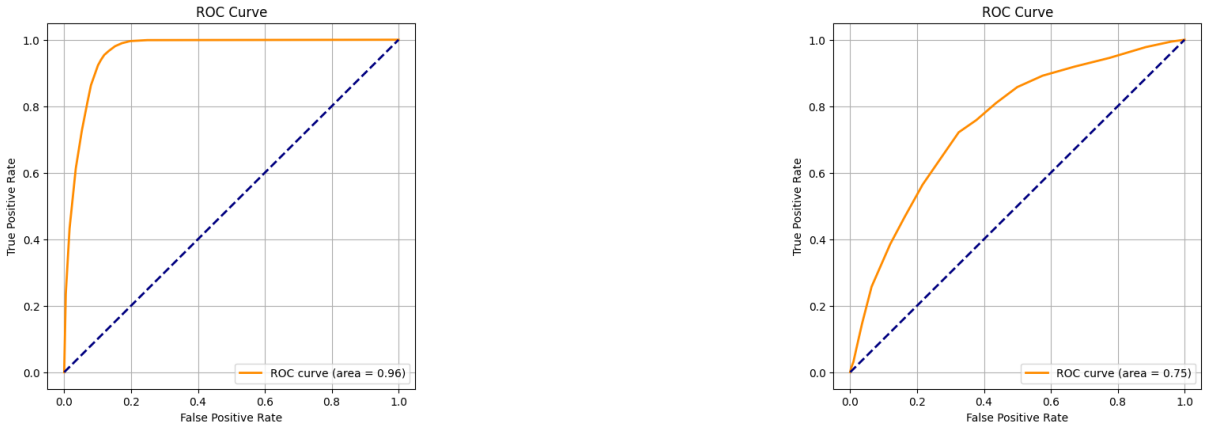
3.1 | Binary classification

1. `isHighRated`: This feature categorizes records based on their ratings. Titles with a rating of 8 or higher are labeled as `1`, while those with a rating below 8 are labeled as `0`. The threshold of 8 was selected to ensure a balanced distribution of the dataset across key features, enhancing the classification model's ability to generalize effectively.

2. `isMovie`: identifies the type of record. Records having `movie` as `titleType` were labeled as `1`, while all other types of content were labeled as `0`.

It is important to note that there categorical variables were not part of the initial dataset analysis presented in the earlier sections of the report, as they were introduced exclusively during the classification phase.

3.1.1 | K-NN



(a) `isMovie` **(b)** `isHighRated`

Figure 3.1: K-NN: ROC curves

In K-Nearest Neighbors (K-NN) algorithm for binary classification, cross-validation was utilized to determine the optimal value of k, which was identified as 15 for `isMovie` and 19 for `isHighRated` targets. The Receiver Operating Characteristic (ROC) curves for both targets (**Figure 3.1**) illustrate the model's performance across various thresholds. The ROC curve for the `isMovie` target confirms the relative

simplicity of this classification task, while the curve for the `isHighRated` target indicates greater difficulty in achieving comparable separability.

For the `isMovie` target, the high performance suggests clear separability and well-defined class boundaries in the dataset, likely due to distinct features distinguishing movies from non-movies. These characteristics are reflected in the achieved metrics: an accuracy of 0.91, a precision of 0.83, a recall of 0.92, and an F1 score of 0.88. The high F1 score indicates a strong balance between minimizing false positives and false negatives, demonstrating that the data provides sufficient discriminatory power.

Conversely, for the `isHighRated` target, the moderate results indicate greater overlap between high-rated and low-rated titles, suggesting less distinct feature patterns or noise within the data, which makes classification more challenging. The model achieved an accuracy of 0.69, a precision of 0.64, a recall of 0.65, and an F1 score of 0.65, reflecting this increased difficulty in separability between classes.

3.1.2 | Naive-Bayes classifier

The Naive Bayes algorithm was evaluated using both the Gaussian and Categorical variants, with the Gaussian model demonstrating superior overall performance. Consistent with the trends observed in the K-NN results, the `isMovie` target exhibited robust performance, whereas the `isHighRated` target produced less favorable outcomes. These disparities can likely be attributed to factors such as class imbalance and non-Gaussian feature distributions, which may have hindered the model's ability to generalize effectively for the `isHighRated` target.

As detailed in **Table 3.2**, the `isMovie` target achieved strong metrics, underscoring the model's effectiveness in distinguishing movies from other types of content. In contrast, the performance for the `isHighRated` target was more moderate, with lower precision and recall values highlighting difficulties in accurately identifying high-rated titles. These challenges may stem from overlapping feature distributions between the two classes, which reduce the model's discriminatory power for this task.

Target	Accuracy	Precision	Recall	F1 Score	ROC AUC
`isMovie`	0.90	0.89	0.90	0.89	0.95
`isHighRated`	0.64	0.67	0.66	0.64	0.72

Table 3.2: performance metrics for Naive Bayes, including ROC AUC scores.

3.1.3 | Decision trees

For the `isMovie` target, the Decision Tree (**Figure 3.2**) indicated strong separability between classes and robust generalization, due to well-defined feature patterns and clear class boundaries within the dataset. After tuning with α pruning, the model achieved optimal results, demonstrating its ability to generalize effectively. Specifically, the model achieved an accuracy of 0.92, a precision of 0.90, a recall of 0.91, an F1 score of 0.92, and a ROC AUC of 0.96.

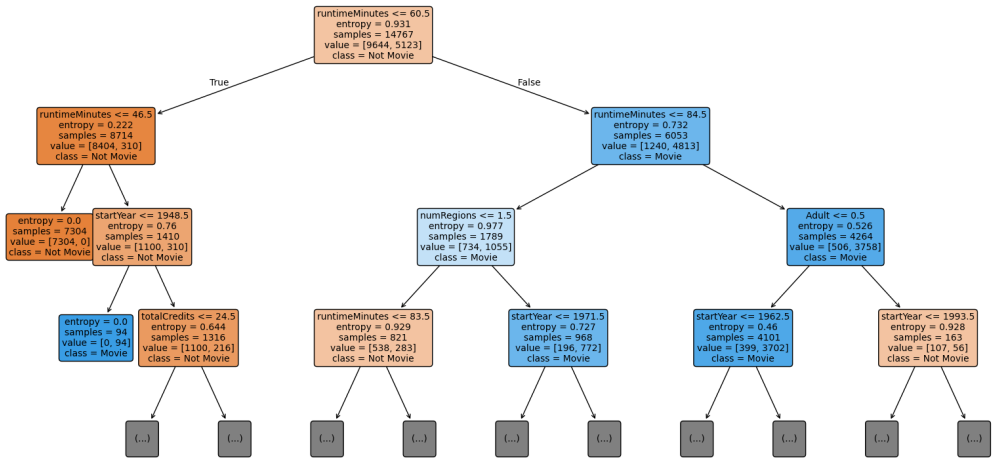


Figure 3.2: optimized Decision Tree for the `isMovie` target, which yielded the best performance.

Conversely, for the `isHighRated` target, the Decision Trees struggled to differentiate between classes, reflecting challenges such as class imbalance, overlapping feature distributions, and weaker correlations between features and the target. These issues resulted in metrics near random performance, with all metrics around 0.51. Interestingly, while α pruning improved the performance for `isMovie`, it further hindered the already weak results for `isHighRated`. A grid search was conducted to determine the optimal hyperparameters for both targets, and the best parameters are presented in **Table 3.3**.

Target	Criterion	Max Depth	Min Samples Leaf	Min Samples Split	α-score
`isMovie`	`entropy`	10	2	10	0.00018
`isHighRated`	`gini`	10	1	5	Excluded

Table 3.3: optimal hyperparameters for Decision Trees for `isMovie` and `isHighRated`.

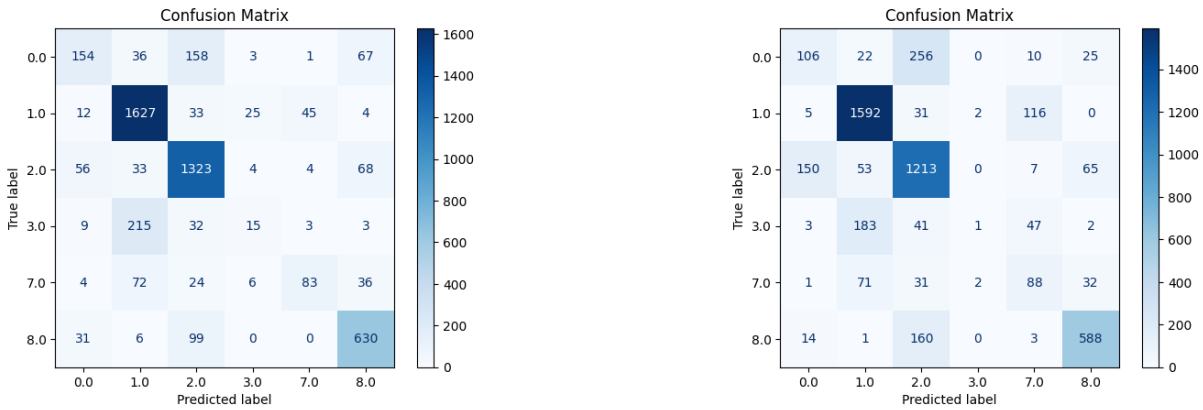
3.2 | Multiclass classification

The `titleType` attribute was selected as the target variable after analyzing the categorical features.

3.2.1 | K-NN and Naive-Bayes classifier

For K-NN and Naive Bayes (Gaussian variant), multiclass classification of the `titleType` target showed similar trends. Poorer performance was observed for `tvSeries` and `tvMovie`, likely due to fewer records. After cross-validation to determine the optimal `k` (9), K-NN achieved Accuracy 0.78, Precision 0.74, Recall 0.78, and F1 Score 0.75 (weighted). Naive Bayes yielded similar results, with an Accuracy 0.73, Precision 0.69, Recall 0.73, and F1 Score 0.70 (weighted). **Figure 3.3**[7] presents the confusion matrices.

[7] 0 (`tvSeries`), 1 (`Movie`), 2 (`tvEpisode`), 3 (`tvMovie`), 7 (`Video`), and 8 (`Short`). Categories 4 (`tvMiniSeries`), 5 (`tvShort`), and 6 (`tvSpecial`) were dropped (see **Section 1.2.2**)



(a) K-NN Confusion Matrix

(b) Naive Bayes Confusion Matrix

Figure 3.3: confusion matrices for the `titleType` target variable: K-NN (left) and Naive Bayes (right).

3.2.2 | Decision trees

To optimize the *Decision Tree* model, both *Grid Search* and *Randomized Search* with cross-validation were used. The best results were obtained with the following parameters: `criterion = gini`, `max_depth = 10`, `min_samples_leaf = 1`, `min_samples_split = 5`. These values limit the complexity of the tree to prevent overfitting and allow for effective node splitting, ensuring each leaf has at least one sample and requiring at least five samples to perform a split.

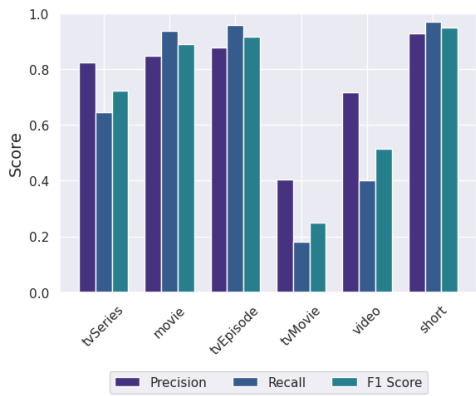


Figure 3.4: performance metrics of the Decision Tree for the multiclass classification of `titleType`.

The optimal pruning parameter was found to be $\alpha = 2.28 \times 10^{-4}$: such a low value indicates a reduced penalty on model complexity, allowing the tree to take advantage of its granted depth. The model achieved good overall results, with a weighted *Accuracy* of 0.86, *Precision* of 0.84, *Recall* of 0.86, and an *F1 Score* of 0.84. However, it still shows difficulties in correctly classifying the *tvMovie* and *Video* classes, likely due to their lower frequency in the dataset and potential feature overlap with other classes.

4 | Regression

4.1 | Univariate and multiple regression

We used regression algorithms for Univariate, Multiple, and Multivariate Regression. After analyzing the variables through Spearman's correlation, we chose for the Univariate Regression, involving one predictor variable and one target variable, the pair `numVotes` and `userReviewsTotal`, as these had the highest

correlation (0.62). Then, for the Multiple Regression, involving multiple predictor variables and one target variable, we added `criticReviewsTotal` and `totalMedia` as the second and third independent variables, as they exhibited an interesting correlation with the dependent variable `userReviewsTotal`. **Table 4.1** presents the results of the various models used for this task: coefficients (mean for Multiple), intercept, R^2 score, Mean Squared Error (MSE), and Mean Absolute Error (MAE).

Algorithm	**Coefficients**	**Intercept**	**R^2**	**MSE**	**MAE**
Univariate Regression {`max_depth:4, min_samples_leaf:4, min_samples_split:2`}					
Linear Regression	0.419	-1.0607	0.530	0.396	0.465
Ridge Regression	0.419	-1.0606	0.530	0.396	0.465
Lasso Regression ($\alpha = 0.1$)	0.414	-1.044	0.529	0.396	0.465
Decision Tree Regressor	-	-	0.561	0.369	0.433
KNN Regressor ($k = 20$)	-	-	0.520	0.404	0.447
Multiple Regression {`max_depth:5, min_samples_leaf:4, min_samples_split:2`}					
Linear Regression	0.244	-0.7261	0.607	0.330	0.424
Ridge Regression	0.244	-0.7261	0.607	0.330	0.424
Lasso Regression ($\alpha = 0.1$)	0.2373	-0.7277	0.607	0.331	0.425
Decision Tree Regressor	-	-	0.616	0.323	0.407
KNN Regressor ($k = 20$)	-	-	0.620	0.319	0.405

Table 4.1: Regression metrics for different algorithms

For Univariate Regression, Linear and Ridge Regression achieved identical performance ($R^2 = 0.530$, MSE $= 0.396$, MAE $= 0.465$), while Decision Tree and K-NN Regressors performed slightly worse. In Multiple Regression, all algorithms showed improved performance, with Decision Tree and K-NN Regressors achieving the highest R^2 values (0.616 and 0.620, respectively) and the lowest MSE and MAE. This suggests that including multiple variables significantly enhances predictive accuracy.

4.2 | Multivariate regression

Multivariate Regression involves multiple predictor variables and target variables. We included `runtimeMinutes` among the predictor variables due to its moderate correlation with `userReviewsTotal` and `totalMedia`, the latter transferred from the predictor variables to the target variables, as it also exhibits a reasonable correlation with `numVotes` and `criticReviewsTotal`. The best parameters chosen through grid search for the Decision Tree Regressor were: `max depth:5`, `min samples leaf:2` and `min samples split:10`.

The results (**Table 4.1**) show that Linear and Ridge Regression performed identically, while Lasso Regression and Decision Tree Regressor yielded slightly worse performance. K-NN Regressor demonstrated the lowest R^2 (0.379) and highest MSE, indicating poorer predictive accuracy among the evaluated models.

Algorithm	R^2	MSE	MAE
Linear Regression	0.399	0.629	0.606
Ridge Regression	0.399	0.629	0.606
Lasso Regression ($\alpha = 0.1$)	0.384	0.647	0.610
Decision Tree Regressor	0.388	0.654	0.593
KNN Regressor ($k = 20$)	0.379	0.661	0.615

Table 4.2: metrics for multivariate regression algorithms.

5 | Pattern mining

To perform a robust association analysis, it was essential to preprocess the data carefully. The categorical variables `titleType` and `genres` were included alongside additional features obtained by transforming continuous variables into categorical ones using a binning technique. This approach grouped continuous values into defined intervals to ensure meaningful distributions and avoid overly imbalanced groupings. The thresholds for these intervals were determined empirically through a trial-and-error process, based on the distribution of the dataset, to create clear and interpretable categories while preserving their relevance.

Variable	Categories (Ranges)
`runtimeMinutes`	`VSLength` (0–15), `SLength` (16–30), `MLength` (31–90), `LLength` (>90)
`startYearDecade`	`Pre-1960`, `1960-1980`, `1980-2000`, `Post-2000`
`rating`	`LRated` (0–5), `MRated` (6–7), `HRated` (8–10)
`numVotes`	`LVotes` (0–30), `MVotes` (31–200), `HVotes` (>200)

Table 5.1: discretization of variables and corresponding categories. Abbreviations: `VS`, `S`, `M`, and `L` stand for `Very Short`, `Short`, `Medium`, and `Long`, respectively. `L`, `M`, and `H` stand for `Low`, `Medium`, and `High`.

5.1 | Frequent patterns extraction

Frequent pattern extraction was performed on the dataset using the Apriori and FP-Growth algorithms. An iterative approach combined with visualization was used to determine the optimal support value, ranging from 0.01 to 0.25. All items in the frequent itemsets were also present in the closed itemsets, resulting in identical outcomes for both Apriori and FP-Growth (**Figure 5.1**). This is likely due to the simplicity of the dataset, which limits the complexity of the patterns. Based on the analysis, a support value of 0.10 was selected, yielding 73 frequent (and closed) itemsets and 44 maximal itemsets.

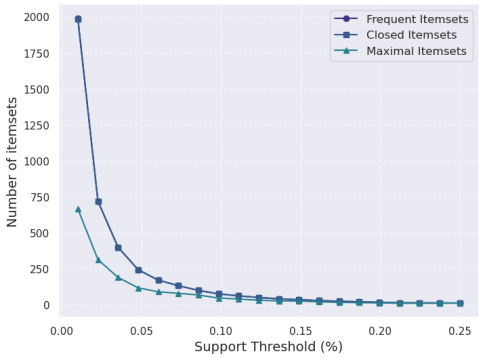


Figure 5.1: Apriori's support values; frequent and closed itemsets are identical.

5.2 | Rules extraction and target prediction

An iterative approach, supported by visualization, was employed to identify the optimal confidence value for the association rules. Based on **Figure 5.2**, a confidence value of 0.6 was selected. The dataset was analyzed using both the Apriori and FP-Growth methods with a support threshold of 0.1. In both cases, 19 association rules were extracted. **Table 5.2** presents the top 10 rules ranked by lift.

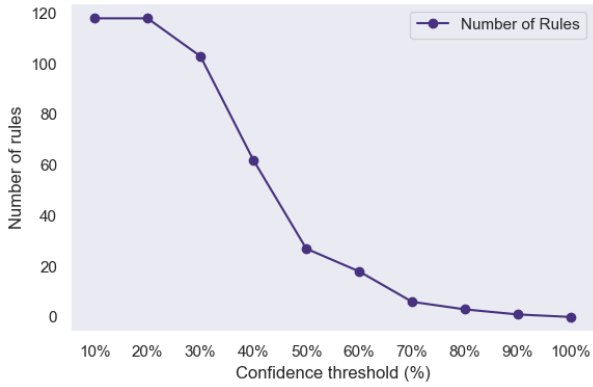


Figure 5.2: association rules per confidence.

For predicting the target variable using Pattern Mining, we focused on the value `HRated`, which aligns with the binary variable `isHighRated` used in Classification (**Section 3.1**). As shown in **Table 5.3**, `HRated` is strongly associated with `tvEpisode` and `SLength`. This result is semantically meaningful, as `tvEpisode` entries are statistically among the shortest categories in the dataset. Furthermore, this finding is partially supported by **Figure 1.2**, which, despite being a high-level analysis, reveals consistently high ratings for `tvEpisode` entries across all decades.

Antecedents	Consequents	Antecedent Support	Consequent Support	Support	Confidence	Lift
(VSLength)	(Short)	0.138003	0.178434	0.126829	0.919028	5.150535
(Short)	(VSLength)	0.178434	0.138003	0.126829	0.710788	5.150535
(TvEpisode, Comedy)	(SLength)	0.125508	0.299269	0.100975	0.804533	2.688329
(SLength, Comedy)	(TvEpisode)	0.132162	0.298659	0.100975	0.764028	2.558193
(LLength)	(Movie)	0.141508	0.348893	0.121851	0.861091	2.468069
(SLength, HRated)	(TvEpisode)	0.184325	0.298659	0.131806	0.715073	2.394279
(SLength)	(TvEpisode)	0.299269	0.298659	0.192909	0.644603	2.158323
(TvEpisode)	(SLength)	0.298659	0.299269	0.192909	0.645918	2.158323
(TvEpisode, HRated)	(SLength)	0.206522	0.299269	0.131806	0.638219	2.132597
(MLength, MRated)	(Movie)	0.197531	0.348893	0.128809	0.652096	1.869043

Table 5.2: Apriori top 10 rules extraction, relationships between antecedents and consequents and respective metrics ranked by lift. The same result was obtained utilizing FP-Growth.

Antecedents	Consequents	Antecedent Support	Consequent Support	Support	Confidence	Lift	Leverage
(TvEpisode)	(HRated)	0.298659	0.432345	0.206522	0.691497	1.599411	0.077398
(SLength, TvEpisode)	(HRated)	0.192909	0.432345	0.131806	0.683254	1.580347	0.048403
(SLength)	(HRated)	0.299269	0.432345	0.184325	0.615920	1.424604	0.054938

Table 5.3: `HRated` association rules ranked by lift.